

PRACTICAL - 10

TASK1: Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
root@DESKTOP-QU0HVJI:~/practical# mkdir practical-10
root@DESKTOP-QU0HVJI:~/practical# cd practical-10
root@DESKTOP-QU0HVJI:~/practical/practical-10# echo "Hello Ubuntu" > original.txt
root@DESKTOP-QU0HVJI:~/practical/practical-10# echo "Hello Ubuntu" > original.txt
root@DESKTOP-QU0HVJI:~/practical/practical-10# stat original.txt
  File: original.txt
  Size: 12          Blocks: 0          IO Block: 4096   regular file
Device: 8,48      Inode: 3314         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: (  0/   root)   Gid: (  0/   root)
Access: 2026-09-28 06:51:49.294700194 +0000
Modify: 2026-09-28 06:51:44.190699612 +0000
Change: 2026-09-28 06:51:44.190699612 +0000
 Birth: 2026-09-28 06:51:44.190699612 +0000
root@DESKTOP-QU0HVJI:~/practical/practical-10# ln original.txt hardlink.txt
root@DESKTOP-QU0HVJI:~/practical/practical-10# echo "New line" >> hardlink.txt
root@DESKTOP-QU0HVJI:~/practical/practical-10# cat original.txt
Hello Ubuntu
New line
root@DESKTOP-QU0HVJI:~/practical/practical-10# ln -s original.txt symlink.txt
root@DESKTOP-QU0HVJI:~/practical/practical-10# ls -li
3314 hardlink.txt 3314 original.txt 3391 symlink.txt
root@DESKTOP-QU0HVJI:~/practical/practical-10# stat symlink.txt
  File: symlink.txt -> original.txt
  Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 3391         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: (  0/   root)   Gid: (  0/   root)
Access: 2026-09-28 06:51:49.294700194 +0000
Modify: 2026-09-28 06:51:44.190699612 +0000
Change: 2026-09-28 06:51:44.190699612 +0000
 Birth: 2026-09-28 06:51:44.190699612 +0000
root@DESKTOP-QU0HVJI:~/practical/practical-10# find . -inum 50825
root@DESKTOP-QU0HVJI:~/practical/practical-10# rm original.txt
root@DESKTOP-QU0HVJI:~/practical/practical-10# ls -li
3314 hardlink.txt 3391 symlink.txt
root@DESKTOP-QU0HVJI:~/practical/practical-10# cat hardlink.txt
Hello Ubuntu
New line
root@DESKTOP-QU0HVJI:~/practical/practical-10# cat symlink.txt
cat: symlink.txt: No such file or directory
```

```
root@DESKTOP-QU0HVJI:~/practical/practical-10# cat hardlink.txt
Hello Ubuntu
New line
root@DESKTOP-QU0HVJI:~/practical/practical-10# cat symlink.txt
cat: symlink.txt: No such file or directory
root@DESKTOP-QU0HVJI:~/practical/practical-10# ln original.txt hardlink.txt
ln: failed to access 'original.txt': No such file or directory
root@DESKTOP-QU0HVJI:~/practical/practical-10# ls -li original.txt hardlink.txt
ls: cannot access 'original.txt': No such file or directory
3314 hardlink.txt
root@DESKTOP-QU0HVJI:~/practical/practical-10# ln -s original.txt symlink.txt
ln: failed to create symbolic link 'symlink.txt': File exists
root@DESKTOP-QU0HVJI:~/practical/practical-10# ls -li
3314 hardlink.txt 3391 symlink.txt
root@DESKTOP-QU0HVJI:~/practical/practical-10# stat symlink.txt
  File: symlink.txt -> original.txt
  Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 3391         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: (  0/   root)   Gid: (  0/   root)
Access: 2026-09-28 06:51:49.294700194 +0000
Modify: 2026-09-28 06:51:44.190699612 +0000
Change: 2026-09-28 06:51:44.190699612 +0000
 Birth: 2026-09-28 06:51:44.190699612 +0000
root@DESKTOP-QU0HVJI:~/practical/practical-10# stat -L symlink.txt
stat: cannot statx 'symlink.txt': No such file or directory
root@DESKTOP-QU0HVJI:~/practical/practical-10#
```

TASK2: Design a program using mmap() to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```
stat: cannot statx 'symlink.txt': No such file or directory
root@DESKTOP-QU0HVJI:~/practical/practical-10# nano task2.c
root@DESKTOP-QU0HVJI:~/practical/practical-10# gcc task2.c -o task2
root@DESKTOP-QU0HVJI:~/practical/practical-10# ./task2
Data written using mmap(): Hello from mmap!
Data read from mmap(): Hello from mmap!
root@DESKTOP-QU0HVJI:~/practical/practical-10# |
```

```
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main() {
    const char *file = "mmap_data.txt";
    char *message = "Hello from mmap!";
    int fd;
    struct stat st;
    char *mapped;

    // 1. Open file with read/write permissions
    if ((fd = open(file, O_RDWR | O_CREAT, 0666)) == -1) {
        perror("open");
        return 1;
    }

    // 2. Set file size to 100 bytes
    if (ftruncate(fd, 100) == -1) {
        perror("ftruncate");
        close(fd);
        return 1;
    }

    // 3. Get file status (size)
    if (fstat(fd, &st) == -1) {
        perror("fstat");
        close(fd);
        return 1;
    }

    // 4. Map file into process virtual memory space
    mapped = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE,
```

```
    // 5. Write data into memory buffer directly
    strcpy(mapped, message);

    // 6. Flush changes from virtual memory to physical disk
    if (msync(mapped, st.st_size, MS_SYNC) == -1) {
        perror("msync");
    }

    // 7. Read and verify written data
    printf("Data written using mmap(): %s\n", mapped);
    printf("Data read using mmap(): %s\n", mapped);

    // 8. Unmap memory and close file descriptor
    if (munmap(mapped, st.st_size) == -1) {
        perror("munmap");
    }

    close(fd);
    return 0;
}
```